

31-AI-CODE-REVIEW.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Quality Framework

Projet : Vitala

1. Objectif

AI-CODE-REVIEW.md définit les règles officielles de revue de code dans Vitala.

Son objectif est de garantir que tout code produit ou modifié respecte :

- l'architecture ;
 - la sécurité ;
 - la qualité ;
 - la maintenabilité ;
 - les performances.
-

2. Principe fondamental

Un code fonctionnel n'est pas forcément un bon code.

Une revue doit vérifier :

``text id="8kq3mr" Est-ce que ça fonctionne ?

+

Est-ce que c'est correctement construit ?

+

Est-ce que ça restera fiable dans le futur ?

3. Quand effectuer une revue

Une revue est obligatoire pour :

- nouvelle fonctionnalité ;
- nouveau module ;
- modification Database ;
- modification API ;
- changement sécurité ;
- refactoring important ;
- avant production.

4. Rôle de l'IA Reviewer

L'IA reviewer doit être critique.

Elle ne doit pas simplement confirmer.

Elle doit chercher :

- erreurs ;
- risques ;
- incohérences ;
- améliorations possibles.

5. Processus officiel de revue

```text id="p4m8vz"  
Code produit

↓

Analyse automatique

↓

Analyse architecture

↓

Analyse sécurité

↓

Analyse qualité

↓

Corrections

↓

Validation

---

## 6. Analyse architecture

Vérifier :

☐ Le code respecte la structure Vitala

☐ Les responsabilités sont séparées

☐ Les modules restent indépendants

☐ Les dépendances sont justifiées

☐ Aucun mélange des couches

---

## 7. Analyse qualité du code

Vérifier :

### Lisibilité

- noms explicites ;
  - fonctions compréhensibles ;
  - structure claire.
- 

### Maintenabilité

- code facilement modifiable ;
  - logique réutilisable ;
  - faible complexité.
- 

### Cohérence

- conventions respectées ;
  - style uniforme ;
  - patterns utilisés correctement.
-

## 8. Analyse sécurité

L'IA reviewer doit vérifier :

### Authentification

- utilisateur correctement identifié.

### Autorisation

- permissions vérifiées.

### Données

- informations sensibles protégées.

### API

- entrées validées.

### Database

- RLS correctement appliqué.
- 

## 9. Analyse Frontend

Vérifier :

- ☐ Composants correctement séparés
  - ☐ Pas de logique métier excessive dans l'UI
  - ☐ Gestion des états complète
  - ☐ Performance acceptable
  - ☐ Accessibilité respectée
- 

## 10. Analyse Backend

Vérifier :

- ☐ Services correctement organisés

- ☐ Validation présente
  - ☐ Erreurs gérées
  - ☐ Logique métier protégée
  - ☐ Dépendances maîtrisées
- 

## 11. Analyse Database

Vérifier :

- ☐ Modèle cohérent
  - ☐ Relations correctes
  - ☐ Contraintes présentes
  - ☐ Index nécessaires
  - ☐ RLS sécurisé
  - ☐ Migration propre
- 

## 12. Analyse API

Vérifier :

- ☐ Convention REST respectée
  - ☐ Contrats stables
  - ☐ Erreurs documentées
  - ☐ Sécurité appliquée
  - ☐ Versionnement respecté
- 

## 13. Analyse performance

Rechercher :

- requêtes inutiles ;

- calculs coûteux ;
  - chargements excessifs ;
  - mauvaise gestion mémoire.
- 

## 14. Classification des problèmes

Chaque problème trouvé doit être classé :

### **CRITIQUE**

Bloque la livraison.

Exemple :

faille sécurité.

---

### **MAJEUR**

Doit être corrigé avant production.

Exemple :

architecture incorrecte.

---

### **MINEUR**

Amélioration recommandée.

Exemple :

nommage.

---

### **INFORMATION**

Suggestion facultative.

---

## 15. Format obligatoire du rapport

L'IA reviewer doit produire :

```text id="4g8r1c" Résumé général :

Score qualité :

Points positifs :

Problèmes trouvés :

Niveau de gravité :

Corrections recommandées :

Validation : ""

16. Score de qualité

Le score peut être évalué sur :

| Domaine | Note |
|---------------|------|
| Architecture | /10 |
| Sécurité | /10 |
| Code | /10 |
| Tests | /10 |
| Documentation | /10 |

17. Interdictions

Une IA reviewer ne doit jamais :

- ✗ valider sans analyse.
 - ✗ ignorer un problème sécurité.
 - ✗ demander une réécriture complète sans justification.
 - ✗ critiquer sans proposer de solution.
-

18. Revue multi-IA

Pour les fonctionnalités critiques :

Utiliser plusieurs rôles :

IA Builder

Crée le code.

IA Reviewer

Analyse.

IA Security

Vérifie les risques.

IA Architect

Vérifie la cohérence globale.

19. Checklist finale Reviewer

☐ Architecture validée

☐ Sécurité validée

☐ Tests validés

☐ Documentation validée

☐ Aucun problème critique

☐ Livraison autorisée

20. Règle finale

La revue de code n'est pas une critique du travail.

C'est un mécanisme de protection de Vitala.

Chaque ligne importante doit mériter sa place dans le système.

Historique

| Version | Modification |
|---------|--------------------------------------|
| V1.0 | Création des règles de revue code IA |

Fin de AI/31-AI-CODE-REVIEW.md